

IA/FIST Lab Equipment Booking Portal

Technical Documentation: Backend, Frontend and Database

Purpose: code base understanding, architecture and future maintenance

Prepared: 24 September 2026

Source: backend-updated-claude.zip and frontend-updated-claude.zip

Contents

1. Introduction	4
1.1 Purpose of this document	4
1.2 What the system does	4
1.3 User roles	4
1.4 Technology stack	5
1.5 Architecture overview	5
2. How the System Works	6
2.1 Registration and guide confirmation	6
2.2 Login and session	6
2.3 Booking rules and booking flow	7
2.4 Booking history and result tracking	8
2.5 Blocking dates	8
2.6 Equipment management and status	9
2.7 Professor portal	9
2.8 Admin functions	9
2.9 Notifications	10
2.10 Scheduled cleanup	10
2.11 Emails sent by the system	10
3. Backend Documentation	11
3.1 Folder structure	11
3.2 Start up sequence	11
3.3 Environment variables (.env)	11
3.4 API reference	12
3.5 Authentication and authorization code	14
3.6 Booking period utility	14
3.7 Email sending	14
4. Frontend Documentation	15
4.1 Folder structure	15
4.2 Application start	15
4.3 API access	15
4.4 Session, roles and navigation	15
4.5 Routes and pages	17
4.6 Shared components	18
4.7 Data display and forms	18
4.8 Files that are not used	18
4.9 Build and hosting	18
5. Database Documentation	19

5.1 Overview	19
5.2 Table: User	19
5.3 Table: Professor	19
5.4 Table: Equipment	20
5.5 Table: Bookings	20
5.6 Table: Results	20
5.7 Other tables	21
5.8 Relationships	21
5.9 Default data and clean up	21
6. Setup and Deployment	22
6.1 Requirements	22
6.2 Backend	22
6.3 Frontend	22
6.4 First time set up order	22
7. Maintenance Guide	23
7.1 Where to change common things	23
7.2 Coding conventions found in the code base	23
7.3 Known issues and recommendations	24

1. Introduction

1.1 Purpose of this document

This document explains how the IA/FIST Lab Equipment Booking Portal is built and how it works. It is written for developers and administrators who will maintain or extend the system. It covers the backend, the frontend and the database in separate sections, and describes the main business rules of the portal. A separate document, the User Manual, explains how each type of person uses the portal.

1.2 What the system does

The portal lets students and researchers of the lab book time slots on shared laboratory equipment. The main functions are:

- Registration of students and researchers using their institute details. The guide (professor) must confirm each new account by email before the account can be used.
- Booking of equipment slots for a two week booking period, with a limit on the number of bookings per equipment for each guide, and a limit on samples per slot shown to the user.
- A booking history page for the user, where the user can also confirm that results were collected.
- An operator area where machine operators see the bookings for their equipment, update the analysis status, block dates for holidays or maintenance, and switch equipment on or off for booking.
- A professor portal where professors, who are registered by the admin, see their students, view the booking history of each student and remove students whose tenure is over.
- An admin area for adding equipment and professors, changing user roles, deleting users and posting notifications.

1.3 User roles

Every account has a numeric role. Students and staff are stored in the User table with the role number below. Professors are stored in a separate Professor table and receive role 5 inside their login token.

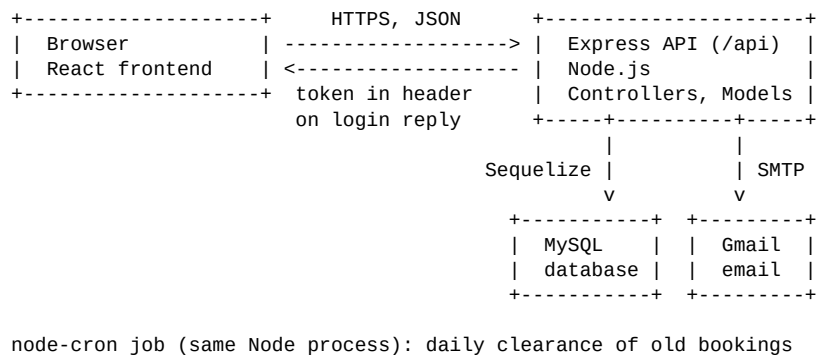
Role	Name	Description
0	User	Default role. Student or researcher who books equipment.
1	Operator (lab technician)	Runs equipment. Sees bookings for own equipment, blocks dates, changes equipment status, adds equipment.
2	TA	Sees the Operator Dashboard menu item only. The list is filled only for equipment where the TA is the assigned operator (see section 7.3).
3	Admin	Manages equipment, professors, users, roles, blocking and notifications.
4	Super admin	Same menu as admin. Created automatically at first start. Its role cannot be changed.
5	Professor	Separate table and separate login page. Sees own students and their booking history.

1.4 Technology stack

Layer	Technology
Frontend	React 18 (Create React App, react-scripts 5), react-router-dom 7, react-auth-kit 3 (session and token storage), axios, MDB React UI Kit, Bootstrap 5, React Bootstrap, Material UI (DataGrid, buttons, dialogs), react-table 7, react-toastify, react-icons, jwt-decode
Backend	Node.js with Express 4, Sequelize 6 (ORM), mysql2, jsonwebtoken (JWT), bcrypt (password hashing), nodemailer (email through Gmail), node-cron (scheduled job), cors, dotenv
Database	MySQL (dialect is read from the DB_DIALECT variable, default mysql)
Email	Gmail SMTP through nodemailer, using EMAIL_USER and EMAIL_PASS

1.5 Architecture overview

The system has three tiers. The browser runs the React application. The React application calls a JSON REST API served by Express under the path /api. Express reads and writes MySQL through Sequelize, and sends email through Gmail. One scheduled job runs inside the same Node process.



2. How the System Works

This section describes the main business flows and the exact rules the code applies. Each flow names the files involved so that a developer can find the code quickly.

2.1 Registration and guide confirmation

- 1 The person opens the Sign Up form (frontend/src/pages/LogInSignUp/LogInSignUp.jsx). The list of guides is loaded from GET /api/professors/getAllProfessors. The guide is chosen from a drop-down list, so the guide must already exist in the Professor table.
- 2 The form sends POST /api/user/createUser (UserController.createUser). The server trims all fields, converts the email to lower case, and validates them (see the validation table below).
- 3 If the email is new, the password is hashed with bcrypt and the User row is created with role 0 and verification set to false.
- 4 The server finds the guide by professorId (the value stored in User.guidId). If no guide email is found, the request returns an error message.
- 5 A confirmation email is sent to the guide in the background. It contains a button linking to FRONTEND_URL/studentConfirmation/{userId}.
- 6 The student is shown the page /verify, which asks the student to contact the guide.
- 7 When the guide opens the link, the page StudentConfirmation.jsx calls POST /api/user/studentConfirmation/{userId}. The server sets verification to true.
- 8 The student can now log in. Before confirmation, login returns the message that the user is not verified yet.

Registration validation rules (backend)

Field	Rule
First name, last name	Letters only, A to Z, no spaces or digits.
Institute ID	Digits only.
Email	Required, unique, stored in lower case. The institute domain check is present in the code but commented out, so any valid email is accepted.
Guide	Required. Holds the professorId of the selected professor.
Mobile number	Must match +91 followed by exactly 10 digits.
Password	At least 6 characters with one lower case letter, one upper case letter, one digit and one special character from @ \$! % * ? & #. Only these character types are allowed.
Confirm password	Checked in the frontend only. Must equal the password.

2.2 Login and session

- Student, operator and admin login: POST /api/user/login. On success the server returns the JWT in the response header Authorization (format: Bearer token). The token contains userId, userName, userEmail, userRole and verification, and expires in 3 days.
- Professor login: POST /api/professors/professorLogin. The token has the same shape, with userRole fixed to 5 and userId equal to the professorId.
- The frontend decodes the token with jwt-decode and stores it with react-auth-kit in localStorage (name: token, expiry 4320 minutes, which is 3 days). The stored user state has userId, userName, userRole, verification and token.

- The menu in the header changes according to userRole (see section 4.4). The PrivateRoute component only checks that a token exists and that verification is true. It does not check the role.
- Password reset for users: POST /api/user/userPasswordResetRequest sends an email with a link to /userPasswordReset/{userId}. The page then calls POST /api/user/userPasswordReset/{userId}. The same rules apply for the new password.

2.3 Booking rules and booking flow

Booking period. Booking is done for a fixed two week window called the active period. The logic is in backend/utils/bookingPeriod.js and is the single source of truth. The frontend asks for it through GET /api/booking/activePeriod.

- Every period is exactly 14 days long.
- A new period opens at 8:30 AM Indian Standard Time on a Wednesday. The periods are counted from an anchor date, 3 January 2024 at 8:30 AM IST, in steps of 14 days.
- IST is treated as a fixed offset of UTC+5:30, so the result does not depend on the server time zone.
- Nothing is stored or scheduled for a period change. The active period is calculated from the current time on every call.
- Old bookings are never moved. Whether a booking belongs to a period is decided from its slotDate.

Screen. The booking table (frontend/src/pages/BookingTable/BookingTable.jsx) shows the 14 dates of the active period as columns and seven fixed time slots as rows: 9AM-10AM, 10AM-11AM, 11AM-12PM, 12PM-1PM, 2PM-3PM, 3PM-4PM and 4PM-5PM. The cell text shows the state of each slot: Book, Selected, Booked, Holiday (Saturday and Sunday), the blocking reason (for a blocked date), or Closed without booking (a slot that has already passed).

Flow.

- 1 On the Home page the user selects one of the equipment items that are working (workingStatus true) and presses Continue. This opens /bookingTable?equipmentId=...
- 2 The table loads three things: the active period, all bookings of the equipment (GET /api/booking/status/{equipmentId}) and all blocked dates (GET /api/blocking/getAllBlocking).
- 3 The user clicks one or more Book cells. The selection is kept in the browser until the user presses Continue.
- 4 Continue sends POST /api/booking/equipmentBookings with equipmentId, userID and the list of selected slots (slotDate, timeSlot, bookingDate).
- 5 The server checks the rules below and creates all bookings inside one database transaction. Either every selected slot is booked or none is.

Rules checked by the server in bookEquipment (BookingController.js)

Check	Result if it fails
At least one booking is sent; slotDate is in YYYY-MM-DD format; timeSlot is present.	400 with a message.
The date is not a Saturday or Sunday.	400: the date is a holiday and cannot be booked.
The date is inside the active period.	400: outside the currently active booking period.
Equipment and user exist.	404.

Check	Result if it fails
Guide quota: bookings already made for this equipment by users of the same guide inside the active period, plus the new bookings, must not exceed the equipment field maxBookingsPerTwoWeeks.	400: Faculty quota exhausted.
Slot is free: the combination equipmentId + slotDate + slotTime must not exist already. The check uses findOrCreate with a row lock inside the transaction.	400: Slot already booked for the date and time. Everything is rolled back.

Important points. The quota is counted per guide and per equipment, so all students of one professor share the same limit for the two weeks. The limit is not per student. The value maxSamples (samples per slot) is shown to users on the Information page but is not counted or enforced by the code. The first user whose request reaches the server takes the slot, which gives the first come first served behaviour.

What is stored. For every booked slot the server writes one row in the Bookings table (status Booked, with a random six digit displayBookingId) and one row in the Results table (operator status: booking reserved; student status: Results not collected). The Results table drives the booking history screens for the user and the operator.

2.4 Booking history and result tracking

- User: page My Booking (StudentsDashBoard.jsx) calls GET `/api/result/findAllBookingForStudent/{userId}`. It shows equipment name, booking date, slot date, slot time, booking ID, operator status and student confirmation. The button Complete Booking calls GET `/api/result/statusUpdateByStudent/{userId}/{resultId}` and sets the student confirmation to Results collected.
- Operator: page Operator Dashboard (OperatorDashBoard.jsx) calls GET `/api/result/findAllBookingForOperator/{userId}`. The server finds all equipment whose operatorUserId equals the operator, and returns the Results rows of that equipment where visibility is true.
- Operator status values: booking reserved (default), in progress, completed, sample issue, technical issue, student was absent on slot time. The operator sets them on the page OperatorUpdating.jsx through POST `/api/result/statusUpdateByOperator`.
- The button Don't Show calls POST `/api/result/updateResultVisibility` and sets visibility to false. The row disappears from the operator list. There is no screen to bring it back.
- No sort order is coded on the server. The operator list appears in database order and the operator can sort by clicking a column heading. The booking ID search box filters the list in the browser.

2.5 Blocking dates

The operator or admin blocks a calendar day for a reason. The screen is EquipmentBlocking.jsx and the API is `/api/blocking`. A blocking record has a month (Jan to Dec), a day number (1 to 31) and one reason: Public Holiday, Equipment Not Working, Professor Booking or External User. The booking table shows the reason text in every cell of that date and does not allow selection.

Note: Blocking is applied by the booking screen only. The booking API does not read the Blocking table. A blocking record has no equipment field and no year, so it applies to all equipment and repeats every year on the same date until it is deleted. See section 7.3.

2.6 Equipment management and status

- Add equipment (admin or operator): AddEquipment.jsx calls POST /api/equipments/equipmentDetails with name, model, guidelines, max samples per slot, max bookings per two weeks and the operator. The server copies the operator name, email and phone number from the chosen user into the equipment row.
- Alter equipment status (admin or operator): UpdateEquipStatus.jsx calls PATCH /api/equipments/updateEquipmentWorkingStatus. A value of false hides the equipment from the selection list on the Home page and lists it as not functioning. Make Visible sets it back to true.
- Delete equipment: DELETE /api/equipments/deleteEquipmentDetails/{id} (button in the equipment list on the Add Equipment page).
- Information page: lists all equipment with guidelines and operator contact details.

2.7 Professor portal

- The admin creates the professor on the Add Professor page. The server adds the prefix Dr. to the first name, hashes the password, saves the Professor row and emails the professor the login email and the temporary password.
- The professor logs in at /ProfessorLogin and lands on /ProfessorDashboard.
- The dashboard calls GET /api/professors/getAllProfessorStudents/{professorId}. It returns every user whose guidId equals the professorId and shows institute ID, name, email and mobile number.
- See Booking history of student opens /professors/viewStudentBooking/{userId}/{professorId}, which calls GET /api/professors/viewStudentBooking/{userId}/{guidId} and shows equipment name, booking date, slot date and slot time.
- Delete student profile calls DELETE /api/professors/deleteProfessorStudent/{studentId}. This is how a professor removes a student whose tenure is over.
- Password change: the email tells the professor to update the password with the forgot password option. The page ProfessorEmailProvide sends POST /api/professors/professorPasswordRequest. See section 7.3 for a fault in the final step.

2.8 Admin functions

Menu item	Page	What it does
Add Equipment	AddEquipment.jsx	Create and delete equipment.
Add Professor	AddProfessors.jsx	Create and delete professors. Email must be an iitr.ac.in address. Password must follow the password rule.
All Users	ListOfUsers.jsx	List all users. Delete a user (admins cannot be deleted from the screen).
Privilege	UserRoleUpdater.jsx	Change a user role to User, Operator, TA, Admin or Professor. The role of a super admin cannot be changed.
Block Booking	EquipmentBlocking.jsx	Create and delete blocked dates.
Alter Equipment Status	UpdateEquipStatus.jsx	Hide equipment from booking or make it visible again.
Notification	Notification.jsx	Post and delete notification messages. Messages scroll on the Home page.

The page /deleteOldData (DeleteRecords.jsx) exists and is routed, but its menu link is commented out in Header.jsx. It calls POST /api/user/deleteOldRecord with a year and deletes users created in that year (except roles 3 and 4) together with their bookings.

2.9 Notifications

Admins post short messages on the Notification page. The Home page reads GET `/api/notification/getAllNotifications` and shows the messages as scrolling text. The three notification endpoints are the only routes that use the token check (`authenticateToken`). The server saves the notification with the `userId` taken from the token and returns only the rows of the logged in user, so at present a message is displayed on the Home page of the account that posted it (see section 7.3).

2.10 Scheduled cleanup

The file `middleware/CronToDistory.js` registers a node-cron job that is meant to run every 24 hours. It reads the newest row of the Clearance table, deletes Bookings created on or before that date, and then empties the Clearance table. A clearance date is created by POST `/api/clear/clearBookingHistoryAutomatically` (there is no screen for it in the frontend). See section 7.3 for a fault in this job.

2.11 Emails sent by the system

Email	Sent to	When	Content
Student confirmation request	The guide	After a student registers	Button with link to <code>/studentConfirmation/{userId}</code>
Password reset (user)	The user	Forgot password	Button with link to <code>/userPasswordReset/{userId}</code>
Professor credentials	The professor	Admin creates a professor	Login email and the temporary password in plain text
Password reset (professor)	The professor	Forgot password on professor page	Link to <code>/ProfessorPasswordReset/{professorId}</code>
Contact request	Fixed address in <code>UserController.js</code>	Contact form submitted	Sender email and message

3. Backend Documentation

3.1 Folder structure

```

backend/
  app.js           Entry point. Creates Express app, mounts /api, syncs DB, starts server
  Router.js        Combines all route files under /api
  package.json     Dependencies. Script: npm start (nodemon app.js)
  .env             Environment variables (do not share or commit)
  config/config.js Sequelize settings for development, test, production
  Auth/Auth.js     authenticateToken: JWT verification middleware
  middleware/
    CheckRole.js   checkRole(roles): role guard middleware (currently unused)
    CronToDistory.js Scheduled cleanup job (node-cron)
  utils/bookingPeriod.js Two week booking period calculation
  models/          Sequelize models, one file per table, plus index.js loader
    User.js Professor.js EquipmentTable.js BookingTable.js ResultTable.js
    BlockingTable.js ClearanceTable.js Notifications.js index.js
  Routers/         URL definitions, one file per area
    UserRoute.js ProfessorsRoute.js EquipmentRoute.js BookingRoute.js
    ResultsRoute.js BlockingRoute.js ClearanceRoute.js Notification.js
  Controllers/     Business logic, one file per area
    userController.js ProfessorsController.js EquipmentController.js
    BookingController.js ResultsController.js BlockingController.js
    ClearanceController.js NotificationsController.js

```

3.2 Start up sequence

- 1 app.js loads models/index.js. That file connects to the database using config/config.js and loads every model file in the models folder. It then runs each model associate function.
- 2 app.js loads middleware/CronToDistory.js, which registers the scheduled job.
- 3 Express middleware is added: urlencoded parser, JSON parser and CORS (all origins are accepted and the Authorization header is exposed so the browser can read the token).
- 4 The router is mounted at /api. A test route /test returns the text Backend is working!
- 5 db.syncTablesInOrder() runs sync with alter true for the tables in this order: User, Equipment, Professor, Booking, Result, Blocking, Clearance, Notifications. After the User table sync, a hook creates the default super admin user if it does not exist.
- 6 The server listens on the PORT from the environment file.

3.3 Environment variables (.env)

Variable	Purpose
PORT	Port the API listens on.
JWT_SECRET_KEY	Secret used to sign and verify login tokens.
DB_USERNAME, DB_PASSWORD, DB_DATABASE, DB_HOST	MySQL connection details.
DB_DIALECT	Sequelize dialect. Defaults to mysql.
DB_DATABASE_TEST	Database name used only when NODE_ENV is test.
EMAIL_USER, EMAIL_PASS	Gmail account and app password used to send emails.
FRONTEND_URL	Public address of the frontend. Used to build the links inside emails. Set it with the protocol (for example https://...). The professor reset email adds http:// in front of it.
NODE_ENV	Optional. development (default), test or production. Selects the block in config/config.js.

The pool size is 5 connections in development and 10 in production. Connection timeout is 60 seconds in development.

3.4 API reference

All paths start with /api. The column Auth shows whether the token check is applied by the server. Only the notification routes apply it.

User routes (Routers/UserRoute.js, userController.js)

Method and path	Function	Purpose	Auth
POST /user/createUser	createUser	Register a user and email the guide.	No
POST /user/login	userLogin	Login. Returns the token in the Authorization header. Requires verification true.	No
PATCH /user/userRole/:userId	userRoleUpdater	Change role. Body: role (0 to 4 as text). Role of a super admin cannot change.	No
DELETE /user/userProfileDelete/:userId	userProfileDelete	Delete a user.	No
PATCH /user/userProfileUpdate/:userId	userProfileUpdate	Update profile fields. Not used by the frontend.	No
GET /user/getSingleUser/:userId	singleUserFinder	Get one user without the password. Not used by the frontend.	No
POST /user/userPasswordResetRequest	userPasswordResetRequest	Send reset email. Always answers with success.	No
POST /user/userPasswordReset/:userId	userPasswordUpdate	Set a new password.	No
GET /user/allUsers	allUserFinder	List all users without passwords.	No
POST /user/studentConfirmation/:userId	confirmUser	Guide confirmation. Sets verification true.	No
POST /user/deleteOldRecord	deleteUsersByYear	Delete users created in a year (not roles 3 and 4) and their bookings.	No
POST /user/contactUs	contactUs	Send the contact form as email.	No

Professor routes (Routers/ProfessorsRoute.js, ProfessorsController.js)

Method and path	Function	Purpose	Auth
GET /professors/getAllProfessors	allProfessorsFinder	List professors without passwords. Used by the sign up form.	No
POST /professors/createProfessorProfile	createProfessorProfile	Create a professor and email the credentials.	No
DELETE /professors/deleteProfessorProfile/:professorId	deleteProfessor	Delete a professor.	No
POST /professors/professorLogin	ProfessorLogin	Professor login. Returns the token in header and body.	No
POST /professors/professorPasswordResetRequest	ProfessorResetRequest	Send reset email.	No
POST /professors/professorsPasswordReset/:professorId	ProfessorPasswordUpdate	Set a new password (see section 7.3).	No
GET /professors/getAllProfessorStudents/:professorId	GetProfessorStudents	Users whose guidId equals the professorId.	No
DELETE /professors/deleteProfessorStudent/:studentId	deleteProfessorStudents	Delete a student.	No
GET /professors/viewStudentBooking/:userId/:guidId	viewStudentBooking	Bookings of a student under a guide, with equipment name.	No

Equipment routes (Routers/EquipmentRoute.js, EquipmentController.js)

Method and path	Function	Purpose	Auth
POST /equipments/equipmentDetails	addEquipment	Create equipment. Body has operatorId.	No
PATCH /equipments/updateEquipmentWorkingStatus	updateEquipmentWorkingStatus	Set workingStatus. Body: equipmentId, workingStatus.	No
GET /equipments/getAllEquipmentDetails	getAllEquipments	List equipment.	No
GET /equipments/getSingleEquipmentDetails/:equipmentId	getEquipmentById	One equipment. Not used by the frontend.	No
DELETE /equipments/deleteEquipmentDetails/:equipmentId	deleteEquipment	Delete equipment.	No
GET /equipments/getOperator	getOperator	Users with role 1, used for the operator drop-down.	No

Booking routes (Routers/BookingRoute.js, BookingController.js)

Method and path	Function	Purpose	Auth
POST /booking/equipmentBookings	bookEquipment	Create bookings (see section 2.3).	No
GET /booking/status/:equipmentId	getBookingById	All bookings of one equipment.	No
GET /booking/getStudentSingleBooking/:userId	getStudentBooking	Bookings of one user. Not used by the frontend, which uses the Result route.	No
GET /booking/activePeriod	getActiveBookingPeriod	Returns periodIndex, startDate, endDate, the 14 dates, and the open and close times.	No
DELETE /booking/cleanOldBooking	clearWeeklyBooking	Deletes every row of the Bookings table. Use with great care.	No

Result, blocking, clearance and notification routes

Method and path	Function	Purpose	Auth
GET /result/findAllBookingForOperator/:userId	AllBookingFinderFromResultForOperator	Visible results of all equipment of the operator.	No
GET /result/findAllBookingForStudent/:userId	AllBookingFinderFromResultForStudent	All results of one user.	No
POST /result/statusUpdateByOperator	statusUpdateByOperator	Body: resultId, operatorStatusConfirmation.	No
GET /result/statusUpdateByStudent/:userId/:resultId	StatusUpdateByStudent	Sets Results collected.	No
POST /result/updateResultVisibility	UpdateVisibility	Body: resultId. Sets visibility false.	No
POST /blocking/createBlocking	BlockingInputs	Body: blockingMonth, blockingNumber, blockingMessage.	No
DELETE /blocking/deleteBlocking/:blockingId	DeleteBlocking	Delete a blocked date.	No
GET /blocking/getAllBlocking	GetAllBlocking	List blocked dates. Returns 404 when there are none.	No
POST /clear/clearBookingHistoryAutomatically	createClearance	Body: clearanceDate such as February 14. Stores a date in the current year.	No
POST /notification/postNotification	postNotification	Body: notificationMessage.	Yes
GET /notification/getAllNotifications	getAllNotification	Notifications of the logged in user.	Yes
DELETE /notification/deleteNotification/:notificationId	deleteNotification	Delete own notification.	Yes

3.5 Authentication and authorization code

- Auth/Auth.js exports authenticateToken. It reads the header Authorization, takes the part after the space, verifies it with JWT_SECRET_KEY and places the decoded payload in req.user. It answers 403 when the token is missing or invalid.
- middleware/CheckRole.js exports checkRole(roles). It compares req.user.userRole with the allowed list and answers 403 if it is not included. It is not used by any route at present.
- The intended use is: router.get(path, authenticateToken, checkRole(["1","3","4"]), handler). A commented example is left in UserRoute.js.

3.6 Booking period utility

utils/bookingPeriod.js exports getActivePeriod, isDateInActivePeriod, isWeekendDate and PERIOD_LENGTH_DAYS. To change when the period starts, change the anchor date ANCHOR_UTC_MS. Keep it on the weekday and time on which periods should open, and keep the period length a multiple of 7 if the opening day must stay the same weekday.

3.7 Email sending

Emails are built inline as HTML strings inside the controllers (UserController.js and ProfessorsController.js). A Nodemailer transport with service gmail is created for each email. The mjml package is installed but not used. To change wording or layout, edit the html string in the matching function.

4. Frontend Documentation

4.1 Folder structure

```

frontend/
package.json      Dependencies. Scripts: start, build, test
public/           index.html, icons, manifest, _redirects (single page app rewrite)
src/
  index.js        Root. Sets up AuthProvider, BrowserRouter, global CSS
  App.js          Renders RoutesConfig and the toast container
  RoutesConfig.js All routes
  Utility/urlInstance.js axios instance with the API base URL
  components/
    Header/Header.jsx Main navigation bar. Menu depends on role
    Footer/Footer.jsx  Footer
    LayOut/LayOut.jsx  Wrapper with optional header and footer
    PrivateRoute/PrivateRoute.jsx Guard for logged in pages
    ConfirmDeleteModal/ Reusable delete confirmation dialog
  pages/          One folder per screen (see 4.5)
  assets/, mediaFiles/ Images

```

4.2 Application start

src/index.js creates the react-auth-kit store with authName token and authType localStorage, and wraps the application in AuthProvider and BrowserRouter. It also imports the MDB and Bootstrap style sheets. App.js renders RoutesConfig and one ToastContainer (top right, closes after 3 seconds).

4.3 API access

All requests go through the axios instance in src/Utility/urlInstance.js. The base URL is written directly in that file. Two other addresses are left as comments (a local address `http://localhost:6370/api` and another hosted address). To run against a local backend, change baseUrl to the local address and change it back before building for production.

Only the notification pages send the token, using the header Authorization taken from the useAuthHeader hook. All other pages send requests without a token because the matching backend routes do not require one.

4.4 Session, roles and navigation

- Login pages call signIn from react-auth-kit with the decoded token data. The hook useAuthUser returns userId, userName, userRole, verification and token anywhere in the application.
- PrivateRoute (used as a wrapper route in RoutesConfig.js) redirects to /login when there is no token and to /verifyLogin when verification is not true.
- Header.jsx holds an object navLinks keyed by role number. It shows the links for the current role, then the common links (Acknowledgement, About, Information), the user name and the Log Out button. Log Out calls signOut and moves to /login.
- Because the guard does not test the role, a logged in user who types the address of another role page in the browser can open that page. Real protection must be added on the backend (section 7.3).

Role	Menu links
0 User	Home, My Booking
1 Operator	Home, Block Booking, Alter Equipment Status, Operator Dashboard, Add Equipment, My Booking
2 TA	Home, Operator Dashboard

Role	Menu links
3 Admin and 4 Super admin	Home, Add Equipment, Add Professor, All Users, Privilege, Block Booking, Alter Equipment Status, Notification
5 Professor	Professor Dashboard
All roles	Acknowledgement, About, Information

4.5 Routes and pages

Path	Page file (src/pages/...)	Access	Purpose
/ and /login	LogInSignUp/LogInSignUp.jsx	Public	Login and sign up in one sliding card.
/signup	LogInSignUp/LogInSignUp.jsx	Public	Same component without the layout.
/about	AboutPage/AboutPage.jsx	Public	About page.
/information	InformationDesk/Information.jsx	Public	Equipment list with guidelines and operator contact.
/ack	Acknowledgment/Acknowledgment.jsx	Public	Acknowledgement page.
/contactForCode	ContactForm/ContactForm.jsx	Public	Contact form. Posts to /user/contactUs.
/ProfessorLogin	ProfessorLogIn/ProfessorLogIn.jsx	Public	Professor login.
/ProfessorEmailProvide	EmailForPassword/EmailForPasswordForProfessors.jsx	Public	Professor forgot password.
/emailProvide	EmailForPassword/EmailForPassword.jsx	Public	User forgot password.
/userPasswordReset/:userId	PasswordUpdater/PasswordUpdater.jsx	Public (from email)	User sets a new password.
/professorPasswordReset/:professorId	PasswordUpdater/ProfessorPasswordUpdater.jsx	Public (from email)	Professor sets a new password.
/studentConfirmation/:userId	StudentConfirmation/StudentConfirmation.jsx	Public (from email)	Guide confirmation. Calls the confirm API when opened.
/verify	verifyRequestPage/VerifyRequest.jsx	Public	Shown after sign up. Asks user to contact the guide.
/verifyLogin	VerifyPage/Verify.jsx	Public	Shown when a logged in user is not verified.
/dashboard	DashBoard/DashBoard.jsx	Logged in	Home. Notifications and equipment selection.
/bookingTable	BookingTable/BookingTable.jsx	Logged in	Slot selection and booking.
/myBookings	StudentDashBoard/StudentsDashBoard.jsx	Logged in	Booking history of the user.
/operatorList	OperatorsDashBoard/OperatorDashBoard.jsx	Logged in	Operator booking list.
/operatorUpdates/:resultId	OperatorUpdating/OperatorUpdating.jsx	Logged in	Set operator status.
/blockBooking	BlockEquipmentSlot/EquipmentBlocking.jsx	Logged in	Manage blocked dates.
/EquipStatusUpdate	UpdateEquipmentStatus/UpdateEquipmentStatus.jsx	Logged in	Hide or show equipment.
/addEquipments	AddEquipment/AddEquipment.jsx	Logged in	Add and delete equipment.
/addProfessors	AddProfessors/AddProfessors.jsx	Logged in	Add and delete professors.
/ListOfAllUsers	ListOfUsersForAdmin/ListOfUsers.jsx	Logged in	List and delete users.
/userRoleUpdate	userRoleUpdater/UserRoleUpdater.jsx	Logged in	Change roles.
/deleteOldData	deleteOldRecords/DeleteRecords.jsx	Logged in (no menu link)	Delete users of a year.

Path	Page file (src/pages/...)	Access	Purpose
/notification	Notification/Notification.jsx	Logged in	Post and delete notifications.
/professorDashboard	ProffesorsDashboard/ProfessorDashboard.jsx	Logged in	Professor student list.
/professors/viewStudentBooking/:userId/:professorId	StudentsBookingDetailForProfessor/StudentsBookingDetail.jsx	Logged in	Student booking history for a professor.
Any other path	PageNotFound/PageNotFound.jsx	Public	Not found page.

4.6 Shared components

- LayOut.jsx: wraps a page with the Header and Footer. The props showHeader and showFooter turn them off.
- ConfirmDeleteModal.jsx: a confirmation dialog with Cancel and Continue. Used by the pages for notifications, blocking, professors, equipment and users.
- Header.jsx and Footer.jsx: see sections 4.4 and 4.1. The footer also has a link to the contact form for code base requests.

4.7 Data display and forms

Tables of records use the Material UI DataGrid (booking history, operator list, users, professors, equipment, blocking and notifications). The booking table uses react-table. Forms use MDB inputs or plain Bootstrap select boxes, with React state for the field values. Messages are shown with react-toastify or as red and green text under the form.

4.8 Files that are not used

The following files are not imported by any active route. They can be kept for reference or removed after the team confirms: components/Header/Header1.jsx (older header with a Clean Old Data link), pages/BookingTable/BookingTable1.jsx and BookingTable1.css (older booking table), pages/AboutPage/AboutPage2.jsx and AboutPage3.jsx, and src/MOCK_DATA.json. Several other files contain large blocks of commented old code.

4.9 Build and hosting

The command `npm run build` creates a static build folder. The file `public/_redirects` contains a rewrite rule so that all paths return `index.html`, which is needed for React Router on static hosts such as Netlify. The application is a pure static site and needs only the API base URL to reach the backend.

5. Database Documentation

5.1 Overview

- Engine: MySQL, accessed through Sequelize. There are no SQL migration files. The tables are created and altered by Sequelize sync (alter true) each time the server starts.
- Eight tables: User, Professor, Equipment, Bookings, Results, Blockings, Clearance, Notifications.
- Table names: the models User, Professor, Equipment and Clearance use freezeTableName, so the tables are named exactly like the models. Booking, Result, Blocking and Notifications use the Sequelize default, which makes the table names plural (Bookings, Results, Blockings, Notifications).
- Every primary key is a UUID generated by Sequelize. Every table has createdAt and updatedAt columns.

5.2 Table: User

Column	Type	Notes
userId	UUID, primary key	Generated automatically.
firstName, lastName	VARCHAR	Letters only (checked in the controller).
instituteld	VARCHAR	Digits only (checked in the controller). Not unique in the database.
guideld	VARCHAR	Holds the professorId of the guide. This is a logical link, not a foreign key.
email	VARCHAR, unique	Must be a valid email.
mobileNumber	VARCHAR	Numeric check. Controller expects +91 and 10 digits.
password	VARCHAR	bcrypt hash.
role	INTEGER, default 0	0 user, 1 operator, 2 TA, 3 admin, 4 super admin.
verification	BOOLEAN, default false	Becomes true after the guide confirms. Login is refused while false.

5.3 Table: Professor

Column	Type	Notes
professorId	UUID, primary key	
firstName, lastName	VARCHAR	First name is saved with the prefix Dr.
email	VARCHAR, unique	IITR address (checked in the controller).
password	VARCHAR	bcrypt hash.
labName, labRoomNumber	VARCHAR, optional	
verification	BOOLEAN, default true	Placed in the login token.

5.4 Table: Equipment

Column	Type	Notes
equipmentId	UUID, primary key	
equipmentName, equipmentModel	VARCHAR	Required.
guidelines	VARCHAR, optional	Usage guidelines. The column length is 255 by default (the form text says 600 characters).
maxSamples	VARCHAR	Samples per slot. Information only.
maxBookingsPerTwoWeeks	VARCHAR	Quota per guide for the active period. Converted with parseInt when used.
operatorName, operatorEmail, operatorPhoneNumber	VARCHAR, optional	Copied from the operator user when the equipment is created.
operatorUserId	UUID, optional	The operator user. Used to find the equipment of an operator. Logical link to User.userId.
workingStatus	BOOLEAN, default true	false hides the equipment from booking selection.
workingStatusMessage	VARCHAR	Default Equipment not functioning. Not used by the screens.

5.5 Table: Bookings

Column	Type	Notes
bookingId	UUID, primary key	
userId	UUID, foreign key to User.userId	Who booked.
equipmentId	UUID, foreign key to Equipment.equipmentId	Which equipment.
guidId	UUID, foreign key to Professor.professorId	Copied from the user at booking time. Used for the quota.
displayBookingId	VARCHAR	Random six digit number shown to users.
bookedDate	VARCHAR	Date on which the booking was made (YYYY-MM-DD).
slotTime	VARCHAR	For example 9AM-10AM.
slotDate	DATE (date only)	The booked day.
bookingStatus	ENUM	Book, Booked, Public Holiday, Equipment Not Working, Professor Booking, External User. Bookings made by users are saved as Booked.

There is no unique index on equipmentId, slotDate and slotTime. Double booking is prevented by the transaction and row lock in the booking code.

5.6 Table: Results

One Results row is created for every booking. It carries the status of the analysis and is the source for the history screens.

Column	Type	Notes
resultId	UUID, primary key	
bookingId	UUID, optional	The booking. Not a foreign key, so it stays if the booking is deleted.
userId	UUID	The user. Association with User is declared in the model.
equipmentId	UUID, foreign key to Equipment	

Column	Type	Notes
guideld	UUID	The guide.
displayBookingId, bookedDate, slotTime, slotDate	VARCHAR	Copies of the booking values. slotDate is text here.
operatorStatusConfirmation	ENUM, default booking reserved	completed, in progress, student was absent on slot time, sample issue, technical issue, booking reserved.
studentConfirmation	ENUM, default Results not collected	Results not collected or Results collected.
visibility	BOOLEAN, default true	false hides the row from the operator list.

5.7 Other tables

Table	Columns	Notes
Blockings	blockingId (UUID, primary key); blockingMonth (ENUM Jan to Dec); blockingNumber (ENUM 1 to 31); blockingMessage (ENUM Public Holiday, Equipment Not Working, Professor Booking, External User)	No equipment and no year. A record applies to all equipment on that day of every year.
Clearance	clearanceId (UUID, primary key); clearanceDate (VARCHAR, YYYY-MM-DD)	Used by the cleanup job. Emptied by the job.
Notifications	notificationId (UUID, primary key); userId (UUID, foreign key association to User); notificationMessage (VARCHAR)	Rows belong to the user who posted them.

5.8 Relationships

From	To	Type	How it is stored
User	Bookings	One to many	Bookings.userId (foreign key)
Equipment	Bookings	One to many	Bookings.equipmentId (foreign key)
Professor	Bookings	One to many	Bookings.guideld (foreign key)
Professor	User (students)	One to many	User.guideld holds professorId (logical link only)
User (operator)	Equipment	One to many	Equipment.operatorUserId (logical link only)
Bookings	Results	One to one	Results.bookingId (logical link only)
User	Results	One to many	Results.userId (association declared)
Equipment	Results	One to many	Results.equipmentId (foreign key)
User	Notifications	One to many	Notifications.userId (association declared)

5.9 Default data and clean up

- At first start, a super admin user (role 4, verified) is created by the afterSync hook in models/User.js if its email does not exist. The email, name and initial password are written in that file. Change the password after the first login and move these values out of the source code (section 7.3).
- There is no seed data for equipment, professors or blocking. The first professors must be added by the super admin, and students can register only after a professor exists.
- Yearly clean up: the page Delete Old Data (POST /user/deleteOldRecord) removes users created in the chosen year and their bookings. Results rows are not removed by it.
- Backup: there is no backup script in the code base. Schedule regular MySQL dumps at the server level.

6. Setup and Deployment

6.1 Requirements

- Node.js (a current long term support version) and npm.
- A MySQL server and an empty database.
- A Gmail account with an app password for sending email.
- A static host for the frontend and a Node host for the backend.

6.2 Backend

- 1 Copy the backend folder to the server and run npm install.
- 2 Create the .env file with the variables from section 3.3. Do not commit it. The .gitignore already lists .env and node_modules.
- 3 Start with npm start (this runs nodemon app.js). For production, use a process manager such as pm2 with node app.js.
- 4 On first start the tables and the super admin user are created. Check the console for the message Tables created in order successfully.
- 5 Open /test on the server address. It should answer Backend is working!

6.3 Frontend

- 1 In frontend/src/Utility/urlInstance.js set baseUrl to the public address of the backend, ending with /api.
- 2 Run npm install, then npm start for development or npm run build for production.
- 3 Upload the build folder to the static host. Keep the _redirects rule so that direct links such as /studentConfirmation/... work.

6.4 First time set up order

- 1 Log in as the super admin (Login page).
- 2 Add professors (Add Professor).
- 3 Have the operators register as normal users, ask their guide to confirm them, then change their role to Operator (Privilege).
- 4 Add equipment and choose the operator (Add Equipment).
- 5 Post any notice (Notification) and add holidays (Block Booking).
- 6 Tell students to register. The system is ready.

7. Maintenance Guide

7.1 Where to change common things

Task	What to change
Change the time slots	The array named data in BookingTable.jsx lists the seven rows. The time text is also read by isPastTimeSlot, so use the same format (for example 9AM-10AM). The server does not validate the time value.
Change the period length or start day	backend/utlis/bookingPeriod.js (PERIOD_LENGTH_DAYS and ANCHOR_UTC_MS). The frontend table has 14 columns written directly (Array(14) and daysOfWeek[13]), so update BookingTable.jsx as well.
Change password rules	The regular expression is repeated in userController.js (createUser and userPasswordUpdate) and ProfessorsController.js (createProfessorProfile and ProfessorPasswordUpdate). Change all four.
Turn on the institute email check for users	Remove the comment marks on the email test in createUser (userController.js).
Add a new blocking reason	Add the text to the ENUM in models/BlockingTable.js, the option list in EquipmentBlocking.jsx and, if it should be a booking status, models/BookingTable.js.
Add a new operator status	Add the text to the ENUM in models/ResultTable.js and to the option list in OperatorUpdating.jsx.
Add a field to equipment	Add the column in models/EquipmentTable.js, accept it in addEquipment, and add it to the form in AddEquipment.jsx and the card in Information.jsx.
Add a new role	Add the number to the list in userRoleUpdater, add an entry in navLinks in Header.jsx, and use it in checkRole.
Change the email text	The HTML strings in userController.js and ProfessorsController.js.
Change the recipient of contact messages	The to address in the contactUs function (userController.js).
Point the frontend to another backend	src/Utility/urlInstance.js.
Add a new page	Create the page in src/pages, add a route in RoutesConfig.js (inside PrivateRoute if it needs login), and add a link in navLinks in Header.jsx.
Add a new API	Add a function in a controller, add the route in the matching file in Routers, and use authenticateToken and checkRole.

7.2 Coding conventions found in the code base

- Controllers return JSON. Errors are usually returned as an errors array, but some functions return message or error, so the frontend reads more than one field.
- Success responses use status 200 or 201. Booking and equipment responses use the key message.
- Validation is done manually inside the controllers with regular expressions.
- There are many console.log lines that print data. Remove them before a production release.
- There is no automated test in either project.

7.3 Known issues and recommendations

The following points were found while reading the code. They are listed so that the maintainers can decide what to fix. They are ordered by importance.

No.	Area	Finding	Suggested action
1	Security: API access	Only the three notification routes check the token. All other routes, including delete user, change role, delete all bookings and list users, can be called by anyone who knows the address. The role guard checkRole exists but is not used. The frontend menu hides pages, but it is not a protection.	Add authenticateToken to every route except login, register, password reset and public lists. Add checkRole for admin and operator actions. Take the user id from the token instead of from the request body or path.
2	Security: default admin	The super admin email and initial password are written in models/User.js. A .env file with live secrets is included in the archive.	Change the password now, move seed values to environment variables, rotate the JWT secret, database password and Gmail app password, and never share the .env file.
3	Security: confirmation link	The guide confirmation link needs no login, and the server does not check who calls it. Anyone who has the link (or the user id) can verify an account. The same is true for password reset links, which contain only the user id.	Use a random one time token with an expiry for confirmation and reset links.
4	Security: other	CORS accepts every origin. The contact form message is placed into HTML without escaping. The professor credentials email contains the password in plain text.	Restrict CORS to the frontend address, escape the message, and send a one time set-password link instead of a password.
5	Professor password reset	ProfessorPasswordUpdate converts the professor id with parseInt (the id is a UUID) and updates where id equals that number. There is no id column, so the reset cannot succeed. The reset email link also puts http:// in front of FRONTEND_URL, unlike the user reset email.	Use the professorId string directly in where { professorId }. Build the link the same way as in the user reset email.
6	Scheduled cleanup	CronToDistory.js imports Bookings, but the model is named Booking. The value is undefined, so the job stops with an error and deletes nothing. There is no screen to set the clearance date. Results rows are not cleaned.	Import Booking, decide whether Results should also be cleaned, and add an admin screen if the feature is needed.
7	Blocking	Blocking is checked only by the booking screen. The API allows a booking on a blocked date. Blocks apply to all equipment and every year, and old blocks stay until deleted. Blocking by maintenance therefore cannot be limited to one machine.	Check the Blocking table in bookEquipment. Add equipmentId (optional) and year to the Blocking table.
8	Equipment status	The API allows booking of equipment whose workingStatus is false. Only the Home page hides it.	Reject the booking in bookEquipment when workingStatus is false.
9	Double booking	There is no unique database index on equipmentId, slotDate and slotTime.	Add a unique index as a second safety.
10	Notifications	The list is filtered by the logged in user, so a notification posted by an admin is displayed only to that admin.	If notices are for everyone, remove the userId filter in getAllNotification and allow every logged in user to read.
11	Database sync	sync with alter true runs at every start. This can change tables unexpectedly in production.	Use Sequelize migrations (sequelize-cli is already installed) and remove alter true.
12	Models	Booking.associate uses the key professorId, but the column is guidId. Some links (Results.bookingId, User.guidId, Equipment.operatorUserID) are not real foreign keys. Deleting a professor who still has bookings depends on the database constraint.	Align the association keys. Test deletion of a professor, a student and equipment on a copy of the data.

No.	Area	Finding	Suggested action
13	Delete user checks	userProfileDelete compares the role with the text 1, but the role is a number, so the server does not protect admins. Only the frontend blocks admin deletion. The old data delete excludes roles 3 and 4 only, while the page text says operators are also excluded.	Compare with numbers and decide the correct list of protected roles.
14	TA role	A TA sees the Operator Dashboard, but the list shows only equipment where the TA is the assigned operator. The operator drop-down lists role 1 users only, so the TA list is normally empty.	Decide the TA role. Either list TAs in the operator drop-down or show bookings of the equipment of their choice.
15	Equipment guidelines	The form says 600 characters, but the column is a VARCHAR of 255.	Change the column to TEXT or limit the form.
16	Operator details	Operator name, email and phone are copied into Equipment when it is created. Later changes to the operator profile do not update them.	Read the operator from the User table at display time.
17	Frontend	The API address is hard coded. The booking table cannot unselect a slot once clicked; the user must reload the page. Old files and commented code remain.	Move the API address to an environment variable (REACT_APP_API_URL). Add an unselect action. Remove unused files.
18	Samples limit	The maximum samples per slot is not stored per booking and not enforced.	Ask for the sample count at booking time if the limit must be enforced.

End of technical documentation.